

11

Lecture 11

**Process Synchronization II: Software-Based
Solutions**

by - Brijesh Shukla
C07: Operating Systems

Join the lecture online on your dashboard.

Today's Lecture Agenda:

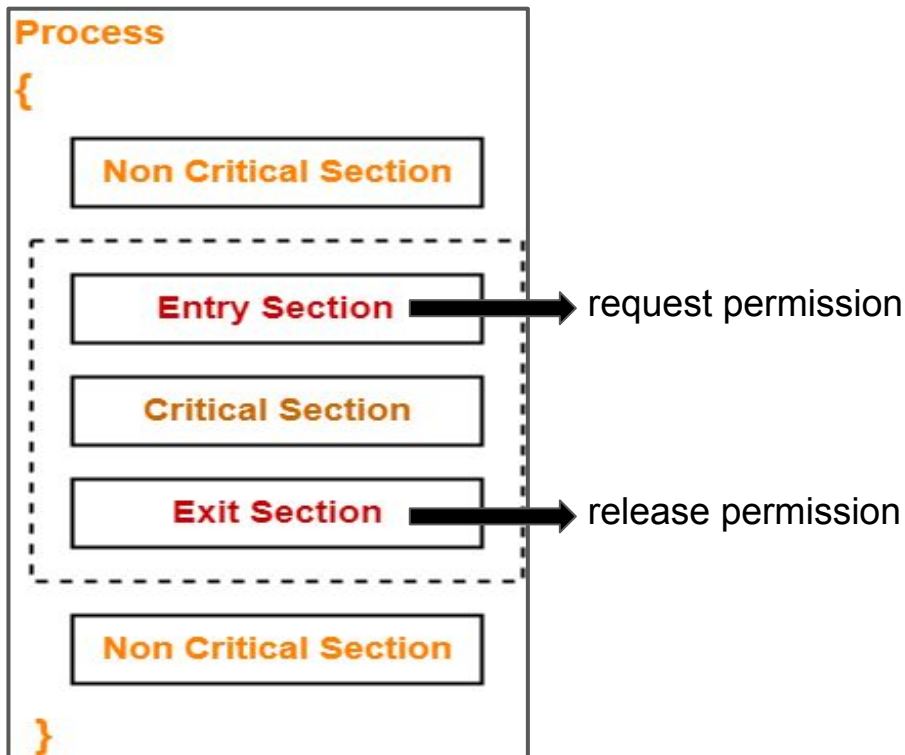
1. Review of Critical Section Requirements
2. Simple Lock Variables and Their Failures
3. Strict Alternation (Turn Variable) Solution
4. The Interest Variable (Individual Interest Flags)
5. Peterson's Algorithm - The First Correct Solution
6. Limitations of Software Solutions

The Journey: From broken attempts to working solutions, understanding why synchronization is fundamentally hard.

Quick Recap - The Critical Section Problem:

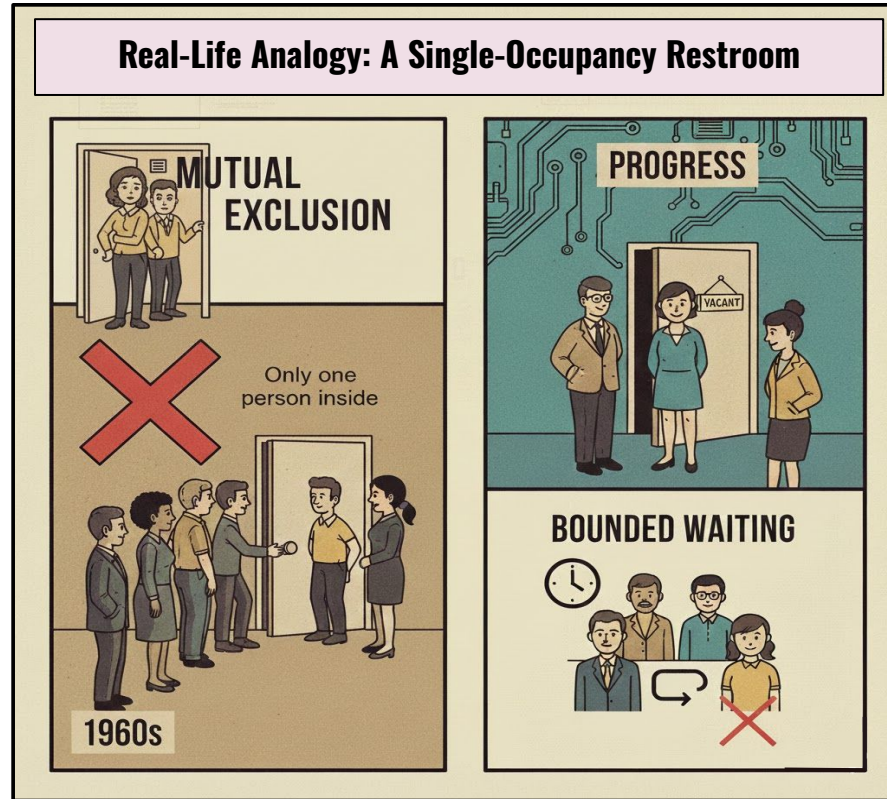
What We Learned in the last lecture:

The critical section is a code segment where processes access shared resources that require protection.



The Challenge: How do we implement entry and exit sections using only software?

Review - The Three Sacred Requirements:



Review - The Three Sacred Requirements:

Any solution **MUST** satisfy all three:



Mutual Exclusion

- **Only ONE process** in critical section at a time
- Prevents data corruption



Progress

- If **CS is free** and processes want to enter, **one MUST** eventually get in
- **No indefinite** postponement
- Only waiting processes decide who enters next

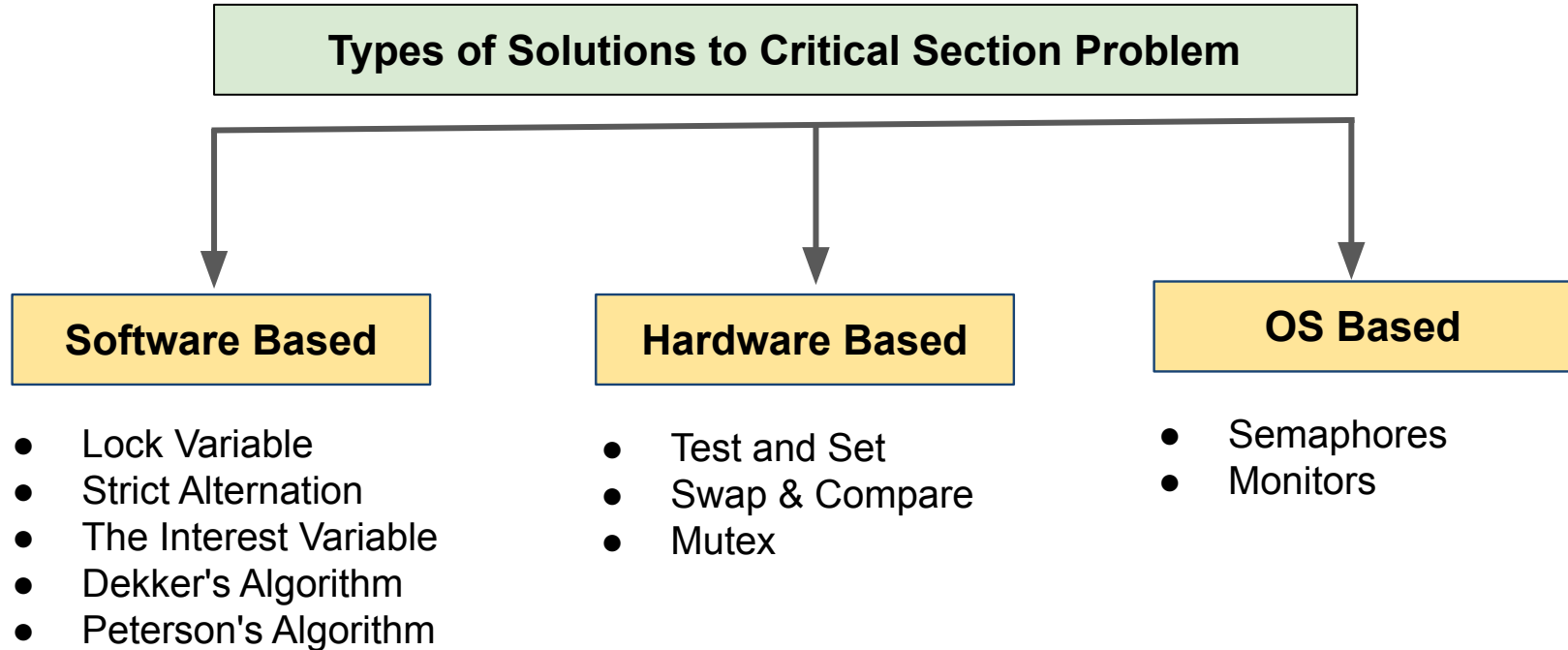


Bounded Waiting

- Every process gets its turn eventually
- **No starvation** (waiting forever)
- Fair access to resources

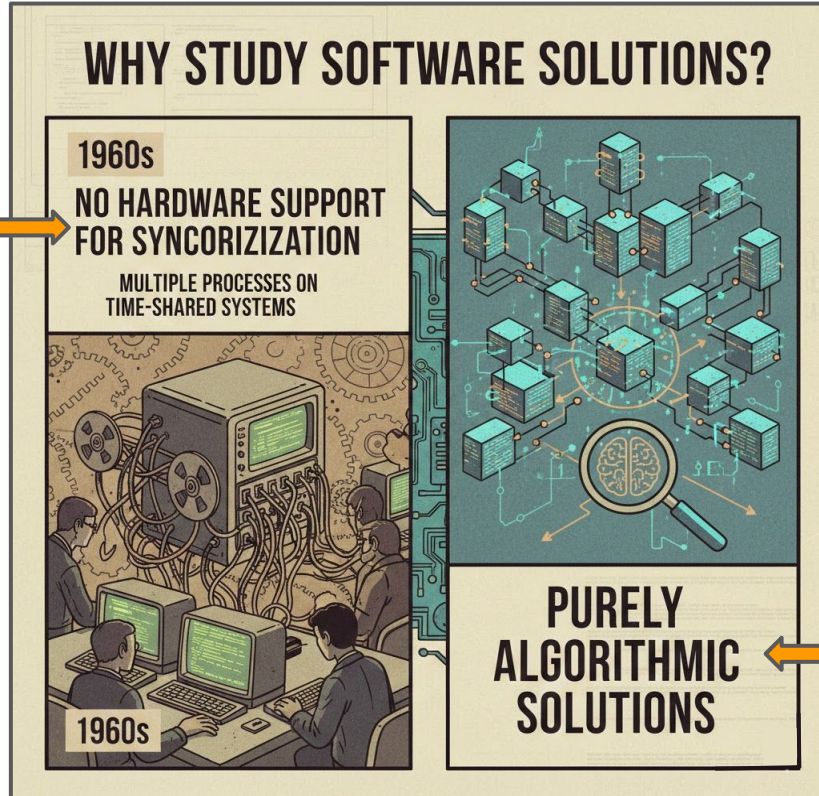
Memory Aid: MPB (Mutual, Progress, Bounded)

Solution to Process Synchronization Problems:



Why Study Software Solutions?

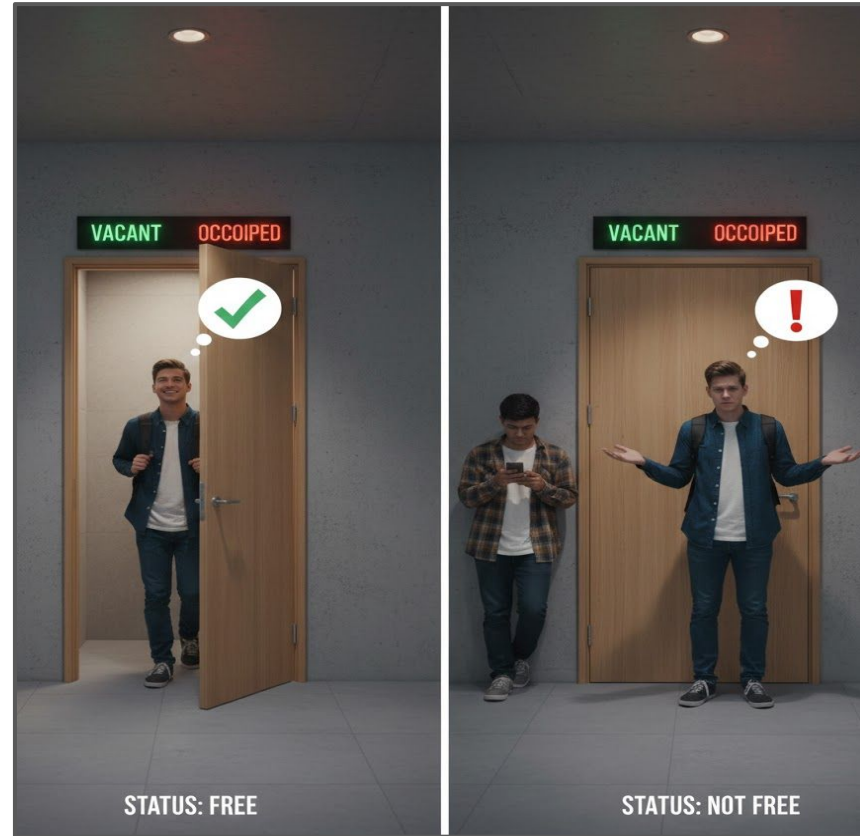
Historical Context:



Needed

The Simple Boolean Lock Variable Solution:

Analogy: Shared Restroom



The Simple Boolean Lock Variable Solution:

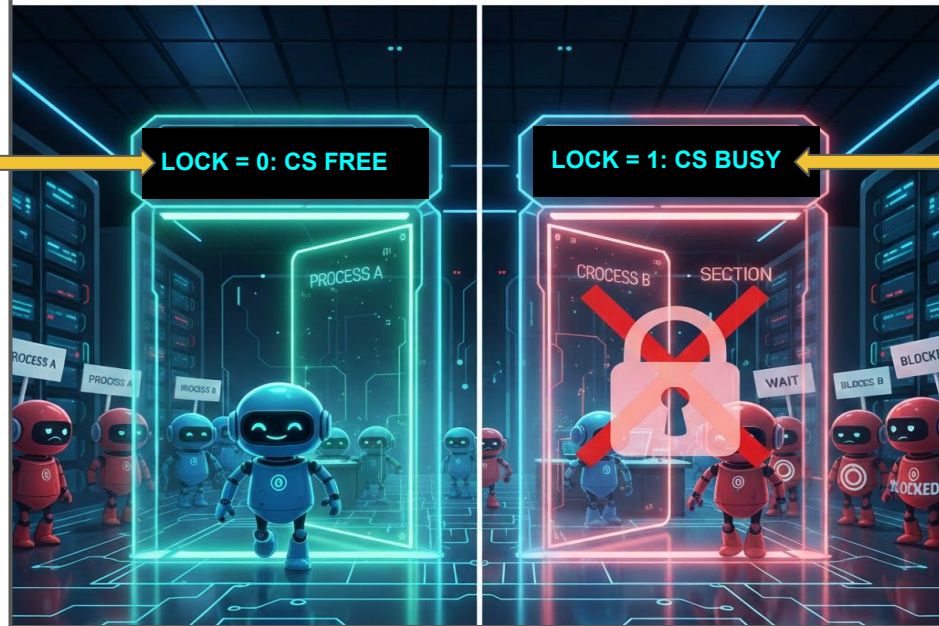
LOCK VARIABLE

For all processes

LOCK = 0: CS FREE

LOCK = 1: CS BUSY

For all processes

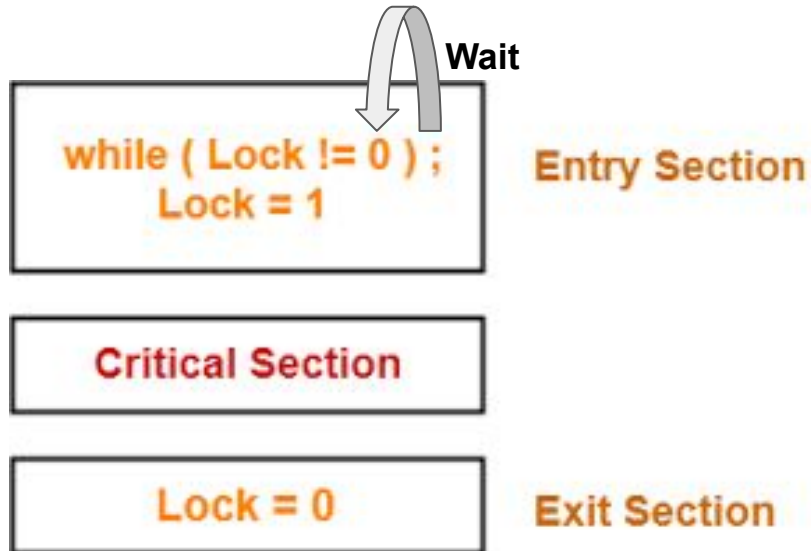


The Simple Boolean Lock Variable Solution:

The most intuitive software attempt uses a single shared boolean lock.

The Logic:

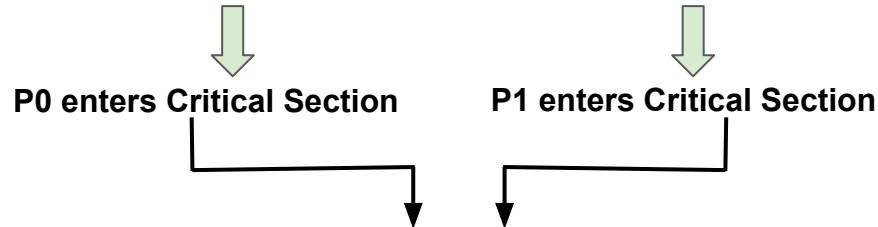
- If **lock == 0**, set it to **1** and enter. Set it to **0** on exit .
- else: wait until **lock == 0**



Why Simple Lock FAILS - Mutual exclusion VIOLATED:

Initially Lock = 0 (Free)

Time	Process P0	Process P1	Lock
t0	while(lock!=0)		0
t1		while(lock!=0)	0
t2	lock = 1		1
t3		lock = 1	1
t4	CRITICAL SECTION	CRITICAL SECTION	1



Both processes in CS! **Mutual exclusion VIOLATED**

Checking for Progress:

Initially Lock = 0 (Free)

Time	Process P0	Process P1	Lock
t0	while(lock!=0)		0
t1		while(lock!=0)	0
t2	lock = 1		1
t3		lock = 1	1
t4	CRITICAL SECTION	CRITICAL SECTION	1

P0 interested
P1 interested

Technically, Progress assumes Mutual Exclusion is already satisfied. If multiple processes can enter together, the "progress" concept becomes meaningless because the critical section isn't being protected at all.

P0 enters Critical Section

P1 enters Critical Section

Both processes in CS! **Mutual exclusion VIOLATED**

✗ No Mutual Exclusion → ✗ No Progress

Checking for Bounded Waiting:

No process should wait **indefinitely** or **unboundedly** long while others keep entering.



Time	Process P0	Process P1	Lock
t0		while(lock!=0) [about to set lock...]	0
t1	while(lock!=0)		0
t2	lock=1 [P0 wins race!]	lock = 1 (too late!) while(lock!=0) [TRUE] ← P1 WAITING	1
t3	// CRITICAL SECTION	← P1 WAITING	1
t4	lock = 0[P0 exits]	while(lock!=0) [TRUE]	0
t5	while(lock!=0)	while(lock!=0)	0
t6	lock = 1 ✓ [P0 RE-ENTERS!]	[P1 still checking...] lock = 1 (too late!) while(lock!=0) [TRUE] ← P1 WAITING	1
t7	// CRITICAL SECTION	← P1 WAITING	1
t8	lock = 0[P0 exits]	while(lock!=0) [TRUE]	0

Why This Violates Bounded Waiting:

P1's Perspective:

- Requested CS at **t0**
- P0 entered CS at times: **t3, t7, t11** (3 times so far)
- **P1 still hasn't entered once!**
- **No bound** on how many times P0 can keep re-entering

P0 interested
P1 interested

Evaluation of The Simple Boolean Lock Variable Solution:

Requirement	Status	Actual Reason
Mutual Exclusion	✗ PRIMARY FAILURE	Race condition allows both processes in CS simultaneously
Progress	✗ Fails (but secondary)	Meaningless when mutual exclusion fails; no proper selection mechanism
Bounded Waiting	✗ FAILS	One process can monopolize (enter repeatedly before other gets a chance)

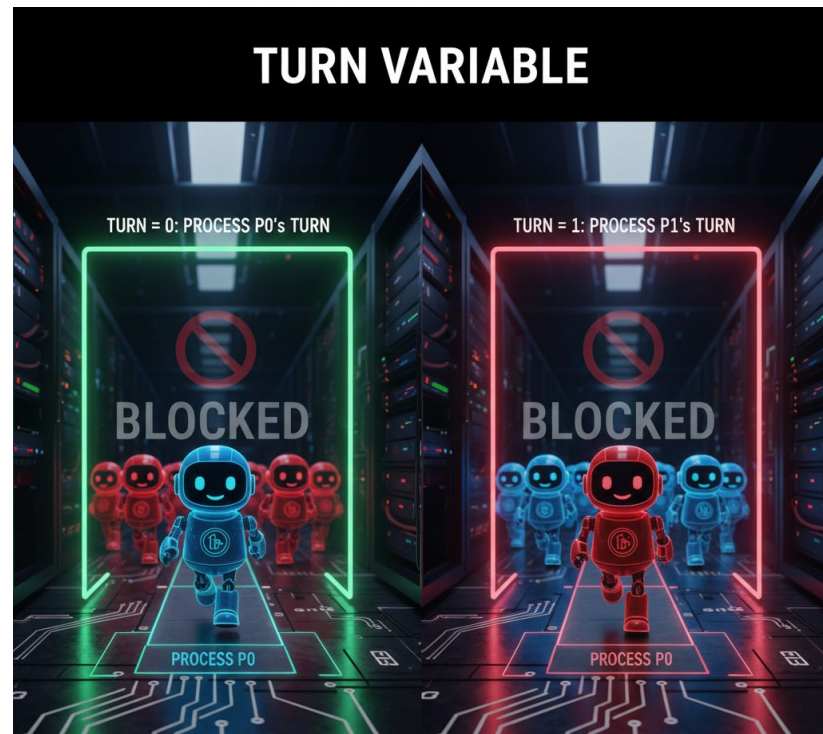
2. Strict Alternation (Turn Variable) Solution:

Using a turn variable to force processes to take turns.

turn = i means "Process i wants to enter the CS".

The Logic:

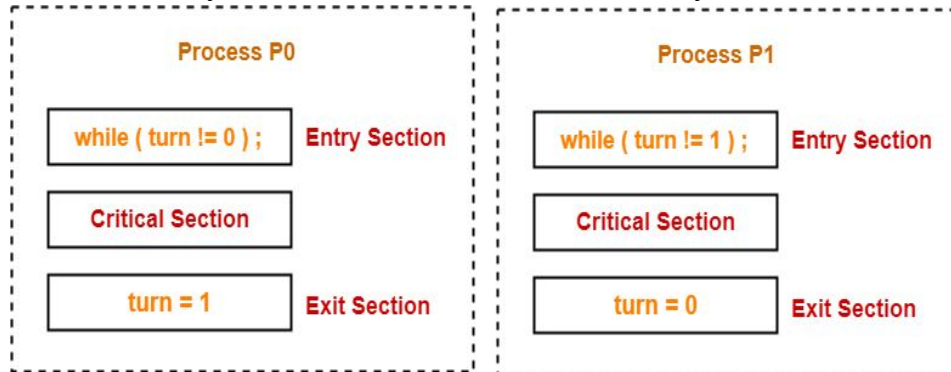
- P0 only enters if **turn == 0**; P1 only if **turn == 1**.
- **Success:** Guarantees Mutual Exclusion.
- **Failure:** Violates Progress



2. Strict Alternation (Turn Variable) Solution:

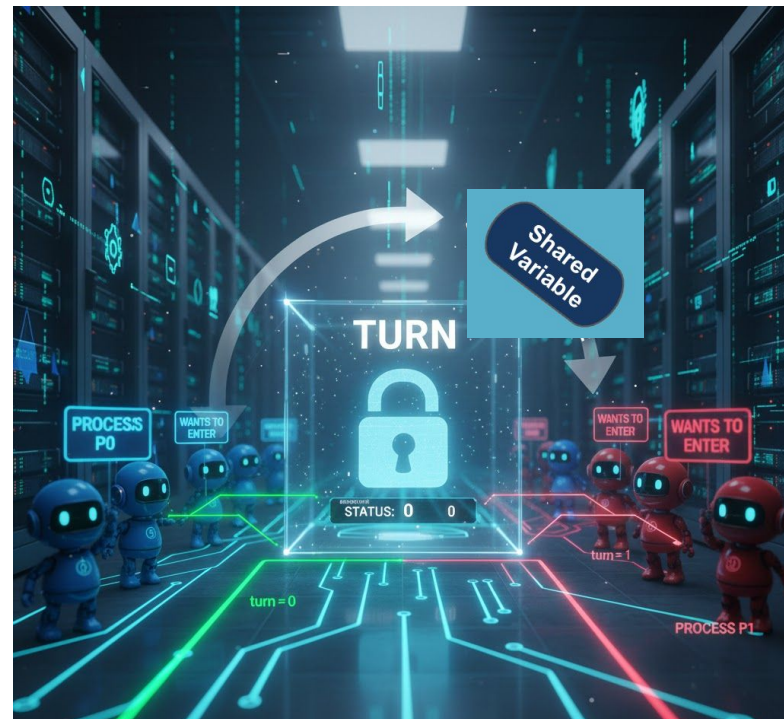
The Logic:

If $\text{turn} = 0$, only P0 can enter



The Logic:

If $\text{turn} = 1$, only P1 can enter



Checking for Progress:

The Analogy Connection: Imagine two neighbors sharing one lawnmower. Neighbor A uses it, then *must* give it to Neighbor B. If Neighbor B goes on vacation and doesn't want to mow, Neighbor A can **never** use the mower again, even though it's sitting idle in the garage



Checking for Progress:

P0 interested
P1 is NOT interested

Initially turn = 0

turn = 0, P0 enters CS, exits, sets turn = 1

P0 wants to enter CS again immediately

P1 is in remainder section (not interested in CS)

CS is FREE but P0 cannot enter!

P0 must wait for P1 to take its turn first



This violates the Progress!

Problem: Process in remainder section (not wanting CS) prevents others from entering. Violates progress requirement.

Checking for Bounded Waiting:

Scenario: P0 wants CS multiple times, P1 wants it once

Initially turn=0

Time	P0	P1	turn
t0	while(turn!=0) ✓	[remainder section]	0
t1	CRITICAL SECTION	[not interested yet]	0
t2	turn = 1	[not interested]	1
t3	[exits CS]		1
	P0 WANTS CS AGAIN!		
t4	while(turn!=0)[TRUE] ← P0 BLOCKED!	[still not interested]	1
t5	← P0 WAITING	[still not interested]	1
t6	← P0 WAITING	[still not interested]	1
t7	← P0 WAITING	[still not interested]	1
∞	← P0 P0 waiting forever	never becomes interested]	1

YES!

✗ It CAN violate Bounded Waiting

Note:

Bound = 1? Only if P1 cooperates! If P1 delays or never comes, **no bound exists**.

Evaluation of Strict Alternation (Turn Variable) Solution:

Requirement	Status	Actual Reason
Mutual Exclusion	✓ YES	Turn can only be 0 or 1 at any time, allowing only one process to pass while($\text{turn} \neq i$) and enter CS.
Progress	✗ FAILS	Uninterested process in remainder section can block interested process. If $\text{turn}=1$ and P1 is not interested, P0 cannot enter even though CS is free.
Bounded Waiting	✗ FAILS	Process must wait for other's turn even if other is not interested. No bound on waiting time if other process delays or never requests CS.

3. The Interest Variable (Individual Interest Flags):



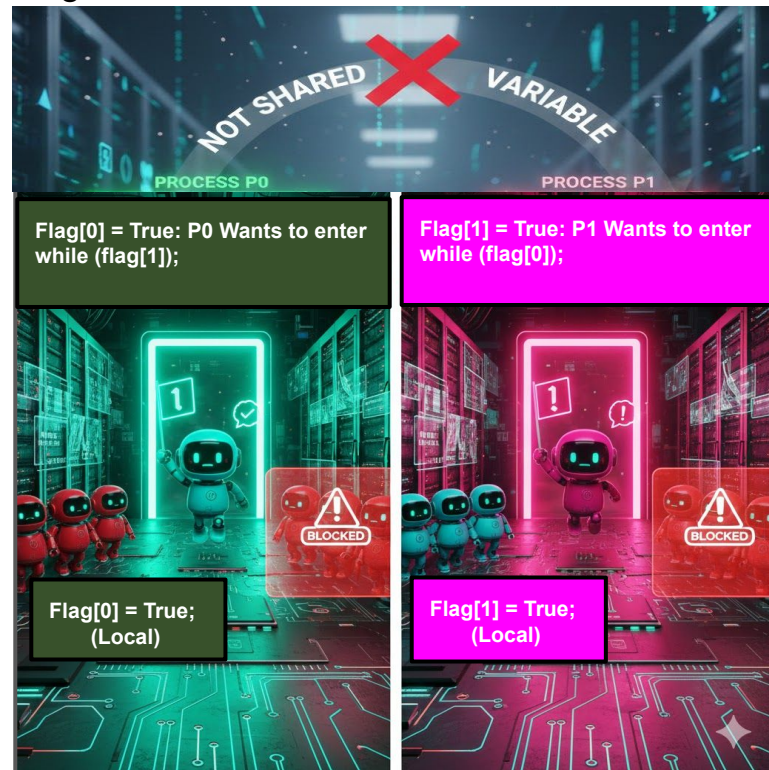
3. The Interest Variable (Individual Interest Flags):

Instead of a "turn," let each process have its own "interest" flag.

$\text{flag}[i] = \text{true}$ means "Process i is ready to enter the CS".

The Logic:

- Set my flag to true, then wait while the other process's flag is **true**.
- **The Failure:** Deadlock (Progress Violation).



3. The Interest Variable (Individual Interest Flags):

Process P0

```
Flag[0] = True; // I'm interested  
while (Flag[1] == True); // Wait while P1 interested
```

CRITICAL SECTION

```
Flag[0] = False;
```

Process P1

```
Flag[1] = True;           // I'm interested  
while (Flag[0] == True); // Wait while P0 interested
```

CRITICAL SECTION

```
Flag[1] = False;
```

The ACTUAL Mutual Exclusion /Progress /Bounded Waiting Violation:

Deadlock Scenario (Mutual Exclusion Failure by Deadlock):

Initially: Flag[0] = False, Flag[1] = False

Time	Process P0	Process P1	Flag[0]	Flag[1]
t0	Flag[0] = True		True	False
t1		Flag[1] = True	True	True
t2	while(Flag[1]==True) [CHECKS: True, WAITS]		True	True
t3		while(Flag[0]==True) [CHECKS: True, WAITS]	True	True
t4	[STUCK IN LOOP]	[STUCK IN LOOP]	True	True
∞	← Deadlock	← Deadlock	True	True
...				



This is DEADLOCK:

- Neither process enters CS
- Both wait forever
- CS remains empty forever
- This **violates the purpose of mutual exclusion** (ensuring CS is used)

No Mutual Exclusion



No Progress

Both processes in CS!
Progress VIOLATED

Due to Deadlock
Bounded Waiting VIOLATED

Here: Neither process can
enter CS at all - infinite wait
time means no bound exists.

Evaluation of The Interest Variable (Individual Interest Flags):

Requirement	Status	Actual Reason
Mutual Exclusion	✗ FAILS	Deadlock scenario: Both set flags True, both check and wait forever. No one enters CS, but also violates the spirit of mutual exclusion as the mechanism fails.
Progress	✗ FAILS	Both interested processes can deadlock. CS is free but neither can enter - selection postponed indefinitely.
Bounded Waiting	✗ FAILS	Deadlock causes infinite waiting time - no bound exists. Processes wait forever without entering CS.

4. Dekker's Algorithm: The First Complete Solution:

Instead of just a "flag" or just a "turn," Dekker's Algorithm combines both to overcome the deadlock seen in the Interest Variable approach. $\text{flag}[i] = \text{true}$ means "Process i is ready to enter the CS".

The Logic:

- **Set Interest:** Process i sets $\text{flag}[i] = \text{true}$ to indicate it ready to enter the Critical Section (CS).
- **Check Conflict:** If the other process also has its flag set to true , check whose turn it is.
- **Yielding:** If it is the other process's turn, Process i will **set its own flag to false** (withdrawing interest) until the turn changes back to them.
- **Mutual Exclusion:** Once the other process lowers its flag or it becomes Process i 's turn, Process i re-sets its flag and enters the CS.

```
//flag[] is boolean array; and turn is an integer  
flag[0] = false  
flag[1] = false  
turn   = 0 // or 1
```

```
P0:  
  flag[0] = true;  
  while (flag[1] == true) {  
    if (turn != 0) {  
      flag[0] = false;  
      while (turn != 0) {  
        // busy wait  
      }  
      flag[0] = true;  
    }  
  }
```

```
// critical section  
...  
turn   = 1;  
flag[0] = false;  
// remainder section
```

```
P1:  
  flag[1] = true;  
  while (flag[0] == true) {  
    if (turn != 1) {  
      flag[1] = false;  
      while (turn != 1) {  
        // busy wait  
      }  
      flag[1] = true;  
    }  
  }
```

```
// critical section  
...  
turn   = 0;  
flag[1] = false;  
// remainder section
```

The Comparison:

- **Why it succeeds:** It overcomes the **Deadlock (Progress Violation)** of the Interest Variable approach because the **turn** variable acts as a tie-breaker.
- **The Improvement:** It ensures **Mutual Exclusion** without the "Strict Alternation" trap, meaning a process doesn't have to wait if the other is not interested.

Evaluation of Dekker's Algorithm:

Requirement	Status	Actual Reason
Mutual Exclusion	✓ PASS	The combined check of flag and turn ensures only one process enters the Critical Section (CS) at a time.
Progress	✓ PASS	By having a process "withdraw" its interest (setting flag = false) when it is not its turn, it prevents the deadlock failure seen in the Interest Variable solution.
Bounded Waiting	✓ PASS	The turn variable acts as a tie-breaker, ensuring that if both processes are interested, they will eventually take turns, and no process is postponed indefinitely.

5. Peterson's Algorithm - The First Correct Solution:

Also called - Simplified Elegance Solution / Humble Solution

Ideology - Combining both:



2. Strict Alternation (Turn Variable) Solution + 3. The Interest Variable (Individual Interest Flags)

Shared Variable

Turn:



Indicates whose turn it is to enter its critical section

Local Variable

Flag[0], Flag[1]:



Used to indicate if a process is ready to enter its critical section

5. Peterson's Algorithm - The First Correct Solution:

Process P0

```
// Indicate that Process P0 wants to enter the critical section
Flag[0] = True;
//Give priority to process P1 in case both want to enter
Turn = j
//Busy wait while P1 is interested/ready and it is P1's turn
while (Flag[1] == True && Turn == j);
```

CRITICAL SECTION

```
// Indicate that Process P0 has exited the critical section
Flag[0] = False;
```

Process P1

```
// Indicate that Process P1 wants to enter the critical section
Flag[1] = True;
//Give priority to process P0 in case both want to enter
Turn = i
//Busy wait while P0 is interested/ready and it is P0's turn
while (Flag[0] == True && Turn == i);
```

CRITICAL SECTION

```
// Indicate that Process P1 has exited the critical section
Flag[1] = False;
```

Checking for Mutual exclusion:

The `Turn` variable acts as a **tiebreaker**!

Initially: Flag[0] = False, Flag[1] = False, Turn = ?

Time	Process P0	Process P1	Flag[0]	Flag[1]	Turn
t0	Flag[0] = True		True	False	?
t1	Turn = 1		True	False	1
t2		Flag[1] = True	True	True	1
t3		Turn = 0	True	True	0
t4	while(Flag[1] && Turn==1) [Flag[1]=True, Turn=0] [Condition FALSE]		True	True	0
t5	ENTERS CS ✓		True	True	0
t6		while(Flag[0] && Turn==0) [Flag[0]=True, Turn=0] [Condition TRUE] ← P1 WAITS	True	True	0



Mutual exclusion ✓ SATISFIES

Checking for Progress:

Time	P0	P1	Flag[0]	Flag[1]	Turn
t0	Flag[0] = True		True	False	?
t1	Turn = 1		True	False	1
t2		Flag[1] = True	True	True	1
t3		Turn = 0	True	True	0
t4	while(Flag[1] && Turn==1) [Turn=0, FALSE]		True	True	0
t5	ENTERS CS ✓	while(Flag[0] && Turn==0)	True	True	0
t6		← P1 WAITS	True	True	0
t7	Flag[0] = False		False	True	0
t8		[Condition now FALSE]	False	True	0
t9		ENTERS CS ✓	False	True	0



Progress ✓ SATISFIES

Note:

- Uninterested processes don't block:** If Flag[j]=False, process i enters immediately
- When both interested:** Turn variable ensures one enters (no deadlock)

Checking for Bounded Waiting:

P0 wants CS twice in a row

Time	P0	P1	Flag[0]	Flag[1]	Turn
	=== FIRST ENTRY ===				
t0	Flag[0] = True		True	False	?
t1	Turn = 1		True	False	1
t2	ENTERS CS ✓		True	False	1
t3	Flag[0] = False [P0 exits]		False	False	1
	=== P1 BECOMES INTERESTED ===				
t4		Flag[1] = True	False	True	1
t5		Turn = 0	False	True	0
t6		ENTERS CS ✓	False	True	0
	=== P0 WANTS CS AGAIN ===				
t7	Flag[0] = True		True	True	0
t8	Turn = 1		True	True	1
t9	while(Flag[1] && Turn==1) [IN CS] ← P0 WAITS	[IN CS]	True	True	1
t10		Flag[1] = False	True	False	1
t11	[Condition FALSE]		True	False	1
t12	ENTERS CS ✓		True	False	1



Bounded waiting ✓ SATISFIES .

Limitations of Software Solutions:

- **Problem 1: Busy Waiting**
- **Problem 2: Modern Hardware Complications**
- **Problem 3: Cache Coherence & Store Buffer Problems**
- **Problem 4: Scalability**



Thank You!